

< 200 CONCRETE OPEN PROBLEMS IN MECHANISTIC INTERPRETABILITY >

200 Concrete Open Problems in Mechanistic Interpretability: Introduction

by **Neel Nanda** 28th Dec 2022

EDIT 19/7/24: This sequence is now two years old, and fairly out of date. I hope it's still useful for historical reasons, but I no longer recommend it as a reliable source of problems worth working on, eg it doesn't at all discuss Sparse Autoencoders, which I think are one of the more interesting areas around today. Hopefully one day I'll have the time to make a v2!

• • •

This is the first post in a sequence called 200 Concrete Open Problems in Mechanistic Interpretability. If you want to learn the basics before you think about open problems, check out [my post on getting started](#).

Skip to [the final section of this post](#)° for an overview of the posts in the sequence

Introduction

Mechanistic Interpretability (MI) is the study of reverse engineering neural networks. Taking an inscrutable stack of matrices where we know *that* it works, and trying to reverse engineer *how* it works. And often this inscrutable stack of matrices can be decompiled to a

human interpretable algorithm! In my (highly biased) opinion, this is one of the most exciting research areas in ML.

There are a lot of reasons to care about mechanistic interpretability research happening. First and foremost, I think that mechanistic interpretability done right can both be highly relevant for alignment. In particular, can we tell whether a model is doing a task well because it's deceiving us or because it genuinely wants to be helpful? Without being able to look at *how* a task is being done, these are essentially indistinguishable when facing a sufficiently capable model. But it also has a lot of fascinating scientific questions - how do models actually work? Are there fundamental principles and laws underlying them, or is it all an inscrutable mess?

It is a fact about today's world that there exist computer programs like GPT-3 that can essentially speak English at a human level, but we have no idea how to write these programs in normal code. It offends me that this is the case, and I see part of the goal of mechanistic interpretability as solving this! And I think that this would be a profound scientific accomplishment.

Purpose

In addition to being very important, mechanistic interpretability is also a very *young* field, full of low-hanging fruit. There are many fascinating open research questions that might have really impactful results! The point of this sequence is to put my money where my mouth is, and make this concrete. Each post in this sequence is a different category where I think there's room for significant progress, and a brainstorm of concrete open problems in that area.

Further, you don't need a ton of experience to start getting traction on interesting problems! I have [an accompanying post](#), with advice on how to build the background skills. The main audience I have in mind for the sequence is people new to the field, who want an idea for where to start. The problems span the spectrum from good, toy, intro problems, to things that could be a serious and impactful research project if well executed, and are accompanied by relevant resources and advice for doing good research. One of the great joys of mechanistic interpretability is that you can get cool results in small models or by interpreting a model that someone else trained. It's full of rich empirical data and feedback loops, and getting your hands dirty by playing around with a model and trying to make progress is a great way to learn and build intuition!

My hope is that after reading this sequence you'll have a clear sense of the contours of this field, where value can be added and how to get started pushing on it, and have a better sense of where the low-hanging fruit *is*.

Disclaimer: As a consequence of this being aimed at people getting into the field, I've tried to focus on concrete problems where I think it'd be easiest to get traction. There are many impactful problems and directions that are less concrete and I haven't focused on. Feel free to [reach out](#) if you have research experience and want more nuanced takes about where to focus!

What's In This Sequence?

Each post corresponds to a (rough) category of open problems. Each post has several sections:

- **Motivation** - Each post begins with a section on motivation - this tries to lay out my thoughts on the area, how I think about it, why it matters, and what problems in there I'm most excited about.
 - I've tried to make these substantial and interesting, and to really convey how I think about each area - I think that just reading each motivation and none of the problems will still give a good overview of the field!
 - Sometimes these include a background or related work section giving optional further context
- **Tips** - Advice for actually working on these problems - research advice, common gotchas and pitfalls, relevant technical knowledge, concrete advice to get started.
 - I've tried to make these substantial and interesting too, and to distill down the right *mindset* for doing research here.
- **Resources** - Resources that I think are particularly important for that category of problems
 - Two worth emphasising are [my mech interp explainer](#), to look up any unfamiliar terms, and to get surrounding context and intuition, and [my TransformerLens library](#) for actually doing mech interp on a GPT-style model. It tries to make the basic operations fast and easy, so you can get started fast and focus on actually doing research!
- **Problems** - A list of the actual problems!
 - **Exciting problems are bolded and starred.** Excitingness is measured, entirely subjectively, by my personal tastes.

- **Problems are rated by difficulty.** Note that these are *very* rough subjective guesses, especially the time estimates. If you're very excited about a problem, that's a great sign you should try working on it!
 - **A** = A good beginner problem, I'd guess that after some skilling up you could do it in a few days to two weeks.
 - **B** = Approachable for beginners, but expect it to be a substantial project, likely at least several weeks to do, longer to do well.
 - But it should be much easier to make some partial progress
 - **C** = Harder or fuzzier, could make a good substantial project but not recommended if you're just starting out. Could easily be a chunky 2-3 month intern project.
 - **D** = Exciting and ambitious, but only a rough idea. You'll need to do significant work scoping out and planning the project.
 - Problems have difficulty ranges (eg B-C) if I think there's an easy version and a hard version, or significant room for how deeply to dig into it (though, really, *every* problem has a significant range of possible depth!).

How to Read This Sequence

- There's a lot of jargon in Mechanistic Interpretability (MI)! I highly recommend having my [MI Explainer](#) up as you read this, and looking up unfamiliar terms in there or googling.
- **Concrete Approaches:**
 - If you don't really know what MI *is*, I recommend reading [Circuits: Zoom In](#) for an overview. You can also just skim through the sequence and see how much you like the vibe.
 - If you're excited about MI, but find the idea of doing research intimidating and don't know where to begin, start with my [Concrete Steps to Get Started](#) post as a guide to the baseline skills and a good starting point.
 - If you have some familiarity with the field and feel ready and excited to jump in and try hacking around, I recommend just reading through whatever sections catch your eye
 - If you want to try hacking around, but feel overwhelmed by choice, I recommend taking a paper you feel excited about, reading the section motivated by building

on that paper, and see whatever catches your fancy. (*Posts will be linked as they come out, see the draft [here](#)*)

- [A list of ML papers I like](#)
- [A Mathematical Framework](#) (walkthrough) -> The Case for Analysing Toy Language Models
- [Interpretability in the Wild](#) (walkthrough) -> Looking for Circuits in the Wild
- [A Toy Model of Superposition](#) -> Exploring Polysemanticity and Superposition
- [A Mechanistic Interpretability Analysis of Grokking°](#) ->
- [In-Context Learning and Induction Heads](#) (walkthrough) -> The Case for Analysing Toy Language Models; Analysing Training Dynamics
- [Softmax Linear Units](#) -> Studying Neurons; Exploring Polysemanticity and Superposition
- [The Image Circuits Thread](#) -> Image Model Interpretability
- [Multimodal Neurons in Artificial Neural Networks](#) -> Studying Neurons; Image Model Interpretability

- **Underlying Mindset:**

- There are two common mindsets I see in people getting into the field:
 - Thinking that even trying research is incredibly hard. That they need to gain a ton of skills first, really gain mastery, and have
 - Thinking that doing research should be easy. That they've failed and aren't cut out for research if they get stuck, make significant mistakes that waste time, or don't make interesting progress.
- I think that both are mistaken!
 - The bar for getting *started* on research isn't that high, especially with the right infrastructure and starting problem. I have an [accompanying post](#) with advice on gaining the baseline skills.
 - But doing *good* research is a real and difficult skill, and especially hard to gain without mentorship.
 - PhDs are designed to teach this mindset and take several years!
- My recommendation is to find a problem you're excited about, *try* to make progress, but be happy if all that happens if you learn a lot.
 - A lot of the skill in ML is practical and intuitive, and best built by playing with models and trying to understand them. Even for the important concepts

best learned by reading papers, these intuitions help to ground things and give context.

- And having a concrete question to make progress on gives a clear goal and direction to focus your learning, and to highlight the skills and knowledge you're missing.
- But also, research is hard, and any concrete step always takes longer than you think! (even after accounting for this rule) It's completely normal to take ages, get confused, and get stuck.
- If all that happens is that you gain skills and experience, but the actual project is a flop, that's still a big win! And if you enjoyed the process, that's a good sign you should continue exploring. I think that the skills gained by actually trying to solve a real problem help form a good foundation to learn more about the field, and attempt more ambitious projects in future.

- **Community:**

- It's a lot easier and more fun to do research with collaborators and mentors, or at least an interested friend you can bounce ideas off!
- There's a [Mechanistic Interpretability Discord](#) where people chat, and there's a fair amount of people interested in solving these problems. The [Eleuther Discord](#) is a good place and more lively, the [#interpretability-general](#) channel is a good place to start
- Jay Bailey has made a [great spreadsheet](#) listing each problem in the sequence. You can write down your contact details if you're working on any of them and want collaborators, see any existing work or reach out to other people on there!
- If you're starting out in research, the problem of finding a mentor is in general hard (and I only have so much capacity, sorry!). Reaching out to researcher is generally a good shout, and reaching out to more junior researchers (eg PhD students, esp at your university if you're a student) often goes well.

Overview of Sequence

A brief overview of each post in this sequence, plus an example problem.

- **The Case for Analysing Toy Language Models°**

- Looking for circuits in one to four layer language models - these are much smaller than real models and so much more tractable, but seem like they'll teach us useful insights! Previous work with two layer attention-only models found induction

heads, which seem to recur in all models studied, and play a crucial role in detecting and learning long-range dependencies in text.

- **Example:** Can you reverse engineer the weights of an interpretable neuron in a one layer language model? (No one has done this, to my knowledge!)
- An example of a (seeming!) base64 neuron in a one layer model:

BASE64 NEURON (IN ONE-LAYER MODEL)

Dataset Examples

<EOT>8efMXtnduFiZVfa
rzKbO9NsZTKzaVMq51cOISAE0otA3QRy/IOzIU+bPRNinePI95g98bBZx5fTC5BqJgSEQGkE6iKQ
0x+azVlqduRYacXVpkybbLbrebPOa6otoXxCQAikBYGaCQTYuPIOs5MfROdC1z+Y/erfRSLRISpN
QiAeBGqahaHbQuQRJXngJvrQi/6sCotYpXW5yay2ifebRcckxRGoiUAY84ji21KsKuh9ZEoxlGwc
Y8W4Bx98MO8sl+3ChQvd0g2kVRK+zs4yD/4r7T/96U/dwleVyrVSOiVw8Re1zJkzx3bs2BG12pbQ

"I'm not going to hold back." Brantley takes important step in comeback:
<https://t.co/O88CkxFOhw> pic.twitter.com/UYcx7cPqk — Jordan Bastian (@MLBastian)
March 17, 2016

Clear acrylic pipe for filtration: <https://amzn.to/30kNQG5>
Clear acrylic pipe products: <https://amzn.to/30hJj7B>
Protein Skimmers of choice: <https://amzn.to/2LDu1Xn>
Finnex LED Aquarium light: <https://amzn.to/2wOlbie>

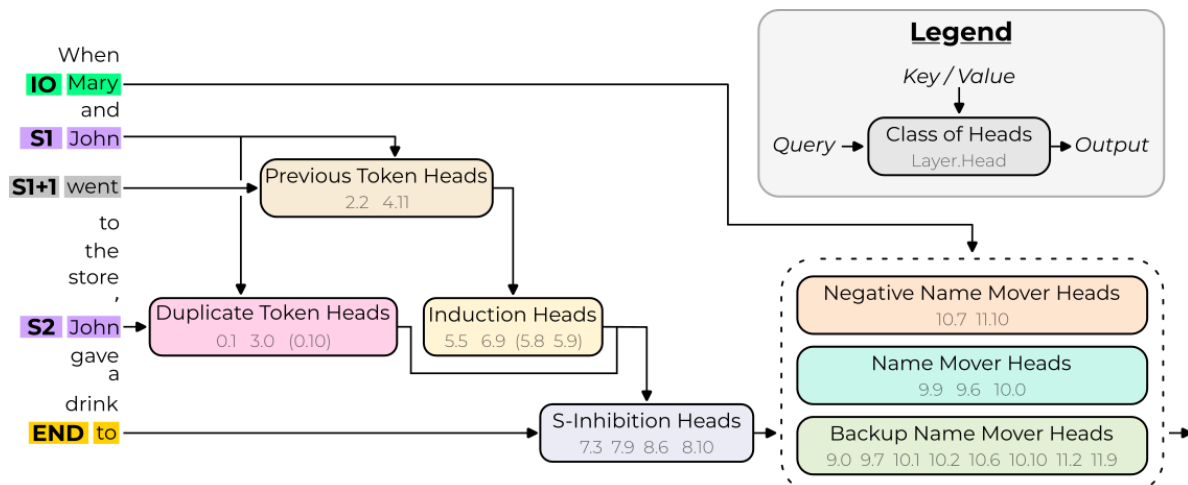
How is "Economic security for all who are unable or unwilling to work" vague? Progressives
are pretty clear about what "economic security" means to them.<https://t.co/CaHliQxszw>
pic.twitter.com/GfxG7ZK8D4 — Jeryl Bier (@JerylBier) February 9, 2019

Logit Weights

+0.17 'w1'	-0.60 'section'
+0.15 'zA'	-0.60 'segment'
+0.14 'mème'	-0.60 'of'
+0.14 'Rp'	-0.61 'exam'
+0.13 'oi'	-0.61 'hatch'
+0.13 'Cc'	-0.61 'dusk'
+0.13 'Tk'	-0.61 'eye'
+0.13 'Hg'	-0.62 'count'
+0.13 'Hz'	-0.62 'time'
+0.12 'mV'	-0.62 'levance'
+0.12 'ZX'	-0.62 'cent'
+0.12 'bW'	-0.63 'circumference'
+0.12 'GQ'	-0.63 'balances'
+0.12 'Cb'	-0.63 'operand'
+0.12 'Nz'	-0.64 'volumes'
+0.12 'rU'	-0.64 'quadrant'
+0.12 'fq'	-0.65 'surface'
+0.12 '+/'	-0.66 'volume'
+0.12 'Dt'	-0.70 'end'
+0.12 'Jy'	-0.72 'compartment'

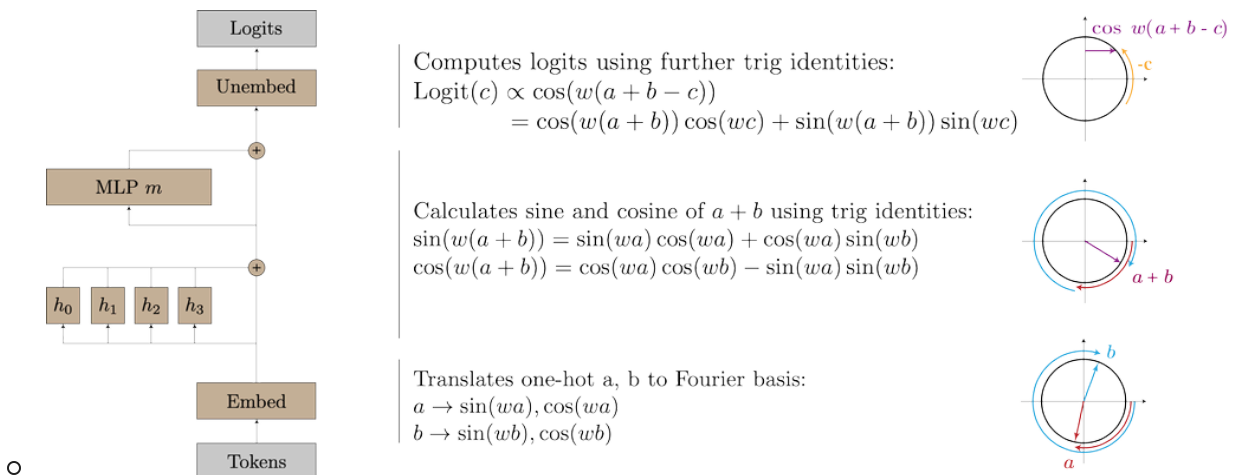
Looking for Circuits in the Wild°

- Looking for circuits in real language models (even tiny ones, like GPT-2 Small). What new challenges come in as we scale up, how confident can we get in our beliefs, and how well do our insights and techniques scale?
- **Example:** The [interpretability in the wild paper](#) reverse engineered the [indirect object identification circuit](#) in GPT-2 Small. Can you use [their code + methods](#) to reverse engineer the corresponding circuit in GPT-Neo Small, and how does the implementation compare?
- The Indirect Object Identification Circuit:



• Interpreting Algorithmic Problems°

- Train and interpret models on algorithmic problems like modular addition. This is much cleaner and has a clear ground truth, so should be much easier (albeit also less useful), but is a great way to practice the key skills and an easy way to test out different techniques and approaches
- **Example:** Can you reverse engineer a one layer transformer doing 5 digit addition? Does the algorithm at all relate to the Fourier Transform based algorithm used to do modular addition?
- The modular addition algorithm:

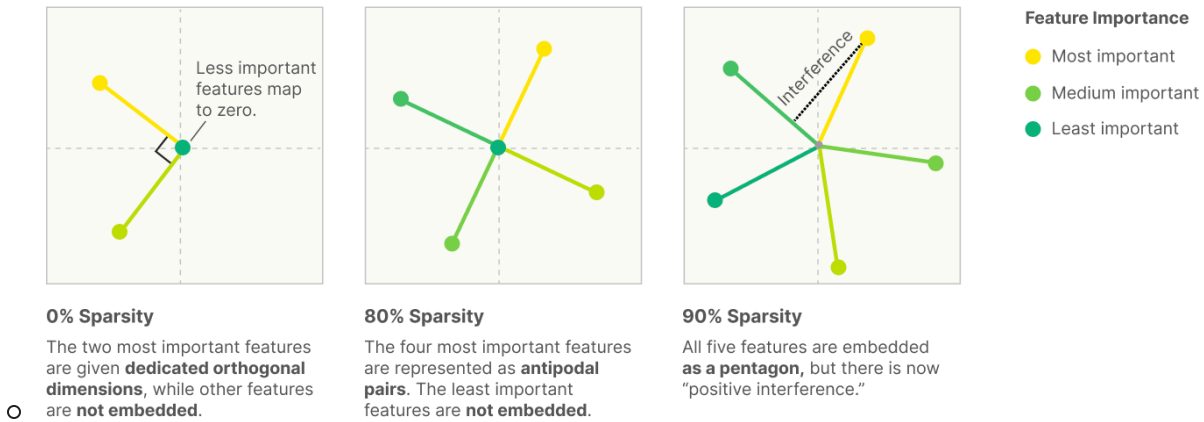


• Exploring Polysemanticity and Superposition°

- Models seem to sometimes engage in superposition - representing more features than they have dimensions by simulating a larger model compressed into a smaller space. What's up with this? How much can we understand this with toy models, and how well do these predictions hold up in practice?
- **Example:** The [Toy Models of Superposition](#) paper predicts that important features are given dedicated dimensions. Positional information is very important in models, and represented directly with positional embeddings in GPT-2. Do these get dedicated dimensions without interference?

As Sparsity Increases, Models Use “Superposition” To Represent More Features Than Dimensions

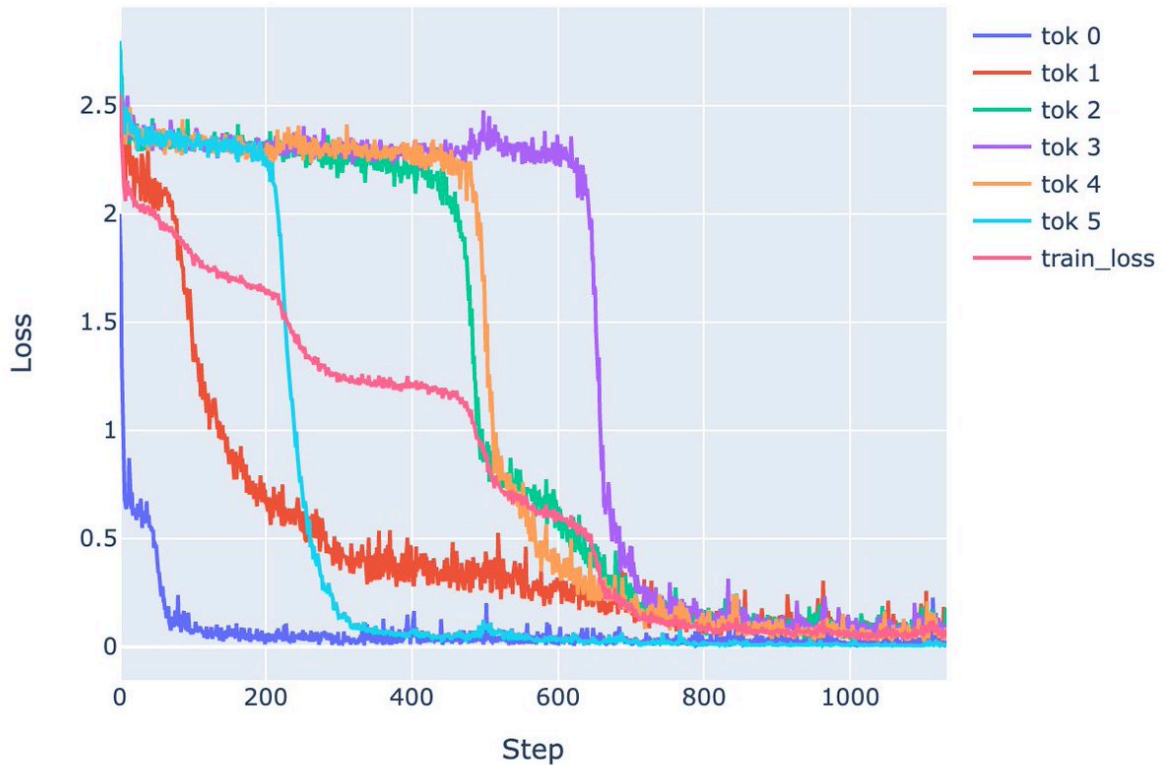
Increasing Feature Sparsity →



• Analysing Training Dynamics°

- When a network is trained, the stochastic gradient descent repeatedly makes small updates to the parameters. After a lot of these steps, we end up with complex models capable of sophisticated behaviour, but what happens in between? Looking at specific circuits often illuminates confusions, such as induction heads which form in a sudden phase transition and cause a small bump in the loss curve.
- **Example:** A toy model trained to do 5 digit addition has a separate phase change per digit. What’s up with this?

Per-digit Loss Curves for 5 digit addition (Infinite Data)

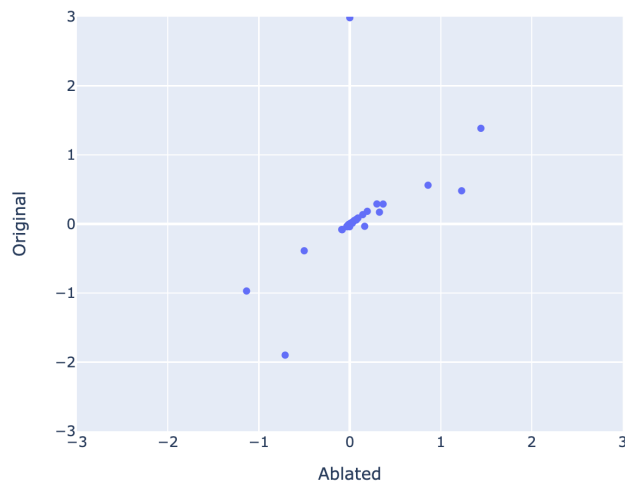


o

- **Techniques, Tooling and Automation**°

- o The core of reverse-engineering networks is forming true beliefs about what's going on inside the network. This is a very hard problem, and developing good techniques and tooling is essential to making progress - from understanding our existing techniques, to finding new ones, to outright automating common steps.
- o **Example:** GPT-2 Small has backup name mover heads - if a certain important head used in indirect object identification is deleted, then a later head takes over to compensate. This makes deleting heads a much worse technique than we'd naively expect! Are there other kinds of backup heads, like backup induction heads?
- o The direct effect on performance from each head, before and after the important head is deleted - note the handful of backup heads that move significantly off the diagonal!

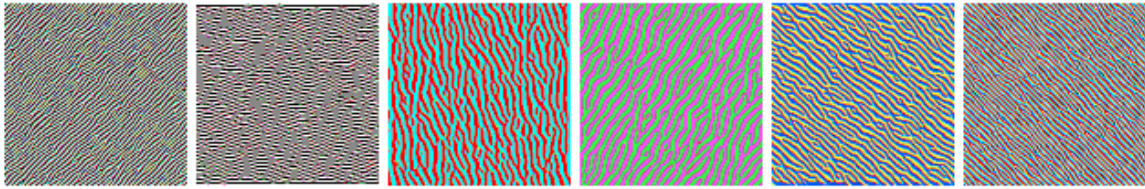
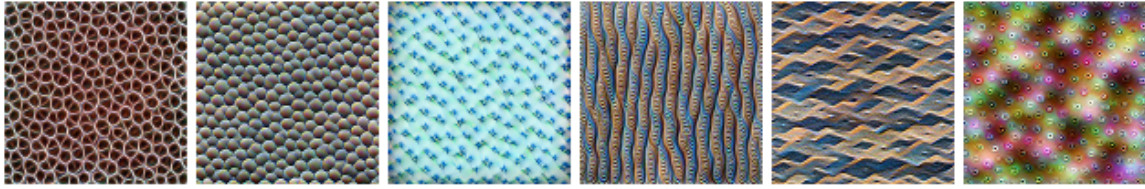
Original vs Post-Ablation Direct Logit Attribution of Heads



○

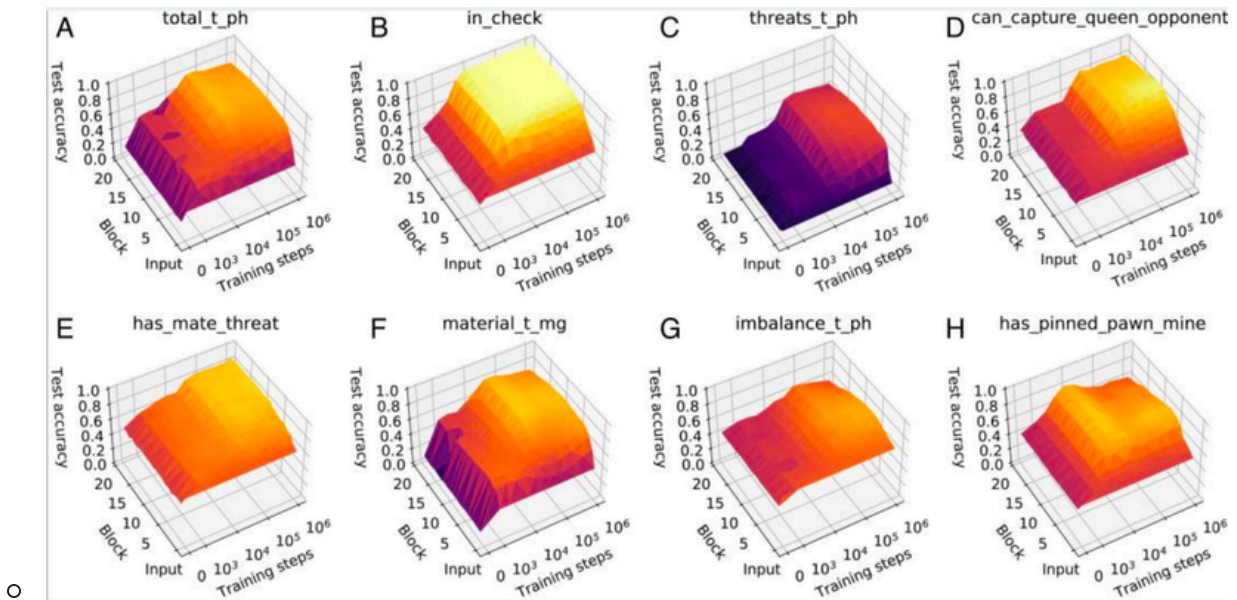
- **Image Model Interpretability**°

- A lot of the early work in mechanistic interpretability was on reverse engineering image classification models. Most of the field now focuses on language models, but I think there's a lot of cool work to build upon, and remaining open problems! I'm particularly interested in knowing how universal our ideas and techniques are - what commonalities are there between language and image data, or transformers and other architectures?
- **Example:** Early layers of image models often contain families of neurons, eg a neuron for curves at angle 0 to the vertical, 45 degrees, 90 degrees, etc. Can you find examples of neuron families in later layers, by looking at interesting neurons in [OpenAI Microscope](#)?
- Examples of the kinds of features found in image classification models:

**Edges** (layer conv2d0)**Textures** (layer mixed3a)**Patterns** (layer mixed4a)**Parts** (layers mixed4b & mixed4c)○ **Objects** (layers mixed4d & mixed4e)

- **Reinforcement Learning**

- Reinforcement learning is the study of how to create agents - models that can act in an environment and form strategies to get high reward. I think there are a lot of deep confusions we have about how RL systems work and how they learn, and that trying to reverse engineer these systems could teach us a lot!
- A cool example of work here is [Tom McGrath's study of chess knowledge acquisition in AlphaZero](#) over training - here are some graphs of how well the model learned different human chess concepts (despite having *no* data on human play!



- **Example:** The Tree Gridworld and Monster Gridworld from [Shah et al](#) are tiny models whose capabilities generalise competently to subtly different environments, but which pursue misgeneralised goals (eg cutting down trees unsustainably fast). Can you reverse engineer them and understand why? And can you predict whether or not this will happen on a new network's weights *before* testing it?

- **Studying Learned Features in Language Models°**

- MLP layers in language models represent $\frac{2}{3}$ of the parameters, yet our understanding of what goes on inside of them is very limited. Our best guess is that they learn to represent features, properties of the input, which sometimes (but not always!) correspond to neurons. And that early layers learn simple features, which get built up into more and more complex features in later layers. But our understanding of *what* features are present is very limited, and I would love to catalogue a bunch!
- In [the SoLU paper](#) they study what a lot of neurons represent and find some fascinating features, notably a complex neuron representing numbers that implicitly describe groups of people:

NUMBER (IMPLICITLY OF PEOPLE)

Dataset Examples

The main banquet room can seat up to 150 guests. This room features neutral decor and the large fireplace adds a warm glow for spring, fall and winter events. The floor to ceiling windows overlook the 9th and 18th holes of our championship golf course.

Star Resorts. In addition to standard hotel rooms, the All-Star Music and Art of Animation Resorts offer two-room Family Suites that can sleep as many as six and provide kitchenettes.

The Legacy Chapel can accommodate up to 70 guests. The Cherish Chapel can accommodate up to 45 guests. The outdoor Terraza overlooks the pool and can accommodate 100 guests.

business in a small garage to become the world's largest manufacturer of "build-it-yourself" component car kits. They employ a full-time crew of about 40 people, and are located in Wareham, Massachusetts (about an hour south of Boston). They make their products right

-
- **Example:** I made a website, [Neuroscope](#), which shows the text that most activates each neuron in several language models. Go and look through middle layer neurons in large models, looking for neurons representing complex concepts. Run a bunch of inputs through the neuron in [an interactive interface](#) until you're confident in this hypothesis

Acknowledgements

This post benefitted greatly from feedback from many people. Thanks to Uzay Girit, Euan Ong, Stephen Casper, Marius Hobhahn, Oliver Balfour, Arthur Conmy, Alexandre Variengien, Ansh Radhakrishnan, Joel Burget, Denizhan Akar, Haoxing Du, Esben Kran, Chris Mathwin, Lee Sharkey, Lawrence Chan, Arunim Agarwal, Callum McDougall, Alan Cooney.

Thanks especially to Jess Smith for inspiring this post and helping write a rough initial draft!

Previous:

Concrete Steps to Get Started in
Transformer Mechanistic Interpretability

5 comments 58 karma

Next:

200 COP in MI: The Case for Analysing
Toy Language Models

2 comments 40 karma

[Log in to save where you left off](#)

Mentioned in

- 52 A List of 45+ Mech Interp Project Ideas from Apollo Research's Interpretability Team
- 24 EIS VI: Critiques of Mechanistic Interpretability Work in AI Safety

14 AISC 2024 - Project Summaries

13 How-to Transformer Mechanistic Interpretability—in 50 lines of code or less!

21 The Engineer's Interpretability Sequence (EIS) I: Intro

[Load More \(5/15\)](#)

[Moderation Log](#)

More from Neel Nanda



[View more](#)

Curated and popular this week

